Correzione Laboratorio 7 - 27.05.2022

Problema 1

Gli script Matlab associati allo svolgimento degli Esercizi 1 e 2 del Problema 1 si trovano nella cartella `CG_FEM_Lab7_Soluzione.zip`.

Esercizio 1

1. L'implementazione della matrice di trasporto globale (sfruttando la `function C_transp_loc.m`) vede la modifica della `function C_matrix2D.m` (che si trova nella cartella `Assembly`). Di seguito si riporta il testo della funzione. In particolare sono stati scritti i comandi alle righe `63-64` del file `C_matrix2D.m`.

2. Le modifiche per risolvere il Problema 1 nel dominio Ω con condizioni di Dirichlet g, funzione f e parametri $\alpha, \boldsymbol{\beta}$ (costanti) assegnati sono state effettuate nelle `function C_main2D.m`, `C_matrix2D.m` e `C_bound_cond2D.m` (nella cartella `BoundaryConditions`). Nella `function C_matrix2D.m` é stata implementata la matrice del problema P alla riga `73`. Nella `function C_bound_cond2D.m` sono state inserite in modo opportuno le condizioni di Dirichlet alla riga `50`. Nella `function C_main2D.m` é stata utilizzata la matrice del problema P per risolvere il sistema lineare alla riga `71`.

3. Per stabilizzare con metodo upwind la soluzione numerica del problema aggiorniamo la matrice P nella `function C_matrix2D.m` alla riga `76`, aggiungendo αPe ad α.

```
1  function [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
2  %% [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
3  %==========================================================================
4  % Solution of the Poisson's problem with linear finite elements
5  % (non homogeneous Dirichlet boundary conditions)
6  %==========================================================================
7  %
8  %    INPUT:
9  %          Dati        : (struct)  see C_dati.m
10 %          nRef        : (int)     refinement level
11 %
12 %    OUTPUT:
```

```
%           errors      : (struct) contains the computed errors
%           solutions   : (sparse) nodal values of the computed and exact
%                         solution
%           femregion   : (struct) infos about finite elements
%                         discretization
%           Matrices    : (struct) fe stiffness and mass matrices
%           Dati        : (struc)  see C_dati.m
%

warning off;
addpath MeshGeneration
addpath FESpace
addpath BoundaryConditions
addpath Assembly
addpath Postprocessing
addpath Errors

%==========================================================================
% LOAD DATA FOR TEST CASE
%==========================================================================

Dati = C_dati(TestName);
Dati.nRefinement = nRef;

%==========================================================================
% MESH GENERATION
%==========================================================================

[Region] = C_create_mesh(Dati);

%==========================================================================
% FINITE ELEMENT REGION
%==========================================================================

[femregion] = C_create_femregion(Dati,Region);

%==========================================================================
% BUILD FINITE ELEMENT MATRICES and RIGHT-HAND SIDE
%==========================================================================

[A,b,T,P] = C_matrix2D(Dati,femregion);

%==========================================================================
% IMPOSE BOUNDARY CONDITIONS
%==========================================================================

[A,b,T,P,u_g] = C_bound_cond2D(A,b,T,P,femregion,Dati);

%==========================================================================
% SAVE DATA in THE STRUCT Matrices
%==========================================================================

```

```
[Matrices] = C_build_out_matrices(A,b,T,P,u_g);

%==========================================================================
% SOLVE THE LINEAR SYSTEM
%==========================================================================

u_0 = P\b;

%==========================================================================
% ASSIGN DIRICHLET BOUNDARY CONDITIONS
%==========================================================================

uh = u_0 + u_g;

%==========================================================================
% POST-PROCESSING OF THE SOLUTION
%==========================================================================

[solutions] = C_postprocessing(Dati,femregion,uh);

%==========================================================================
% ERROR ANALYSIS
%==========================================================================

[errors] = C_compute_errors(Dati,femregion,solutions);
```

```
function [A,f,T,P]=C_matrix2D(Dati,femregion)
%% [A,f,M] = C_matrix2D(Dati,femregion)
%==========================================================================
% Assembly of the stiffness matrix A, rhs f and transport matrix T
%==========================================================================
%    called in C_main2D.m
%
%    INPUT:
%          Dati        : (struct)  see C_dati.m
%          femregion   : (struct)  see C_create_femregion.m
%
%    OUTPUT:
%          A           : (sparse(ndof,ndof) real) stiffness matrix
%          f           : (sparse(ndof,1) real) rhs vector
%          T           : (sparse(ndof,ndof) real) transport matrix
%          P           : (sparse(ndof,ndof) real) problem matrix
%

warning off;
addpath FESpace
addpath Assembly

fprintf('==========================================================\n')
fprintf('Assembling matrices and right hand side ... \n');
fprintf('==========================================================\n')

```

```

% connectivity infos
ndof        = femregion.ndof;
nln         = femregion.nln;
ne          = femregion.ne;
connectivity = femregion.connectivity;


% shape functions
[basis] = C_shape_basis(Dati);

% quadrature nodes and weights for integrals
[nodes_2D, w_2D] = C_quadrature(Dati);

% evaluation of shape bases
[dphiq,Grad] = C_evalshape(basis,nodes_2D);


% Assembly begin ...
A = sparse(ndof,ndof);
T = sparse(ndof,ndof);
f = sparse(ndof,1);

for ie = 1 : ne

    iglo = connectivity(1:nln,ie);

    % Determinant of the Jacobian of the map and physical coordinates
    [BJ, pphys_2D] = C_get_Jacobian(femregion.coord(iglo,:), nodes_2D, Dati.
   MeshType);

    %==================================================================%
    % STIFFNESS MATRIX AND TRANSPORT MATRICES
    %==================================================================%
    [A_loc] = C_lap_loc(Grad,w_2D,nln,BJ);
    A(iglo,iglo) = A(iglo,iglo) + A_loc;

    [T_loc] = C_transp_loc(dphiq,Grad,w_2D,nln,BJ,Dati.beta);
    T(iglo,iglo) = T(iglo,iglo) + T_loc;

    %==================================================
    % FORCING TERM -- RHS
    %==================================================
    [load] = C_loc_rhs2D(Dati.force,dphiq,BJ,w_2D,pphys_2D,nln);
    f(iglo) = f(iglo) + load;
end

P = Dati.alpha * A + T;

if Dati.stabilization == "Y"
    P = P + (norm(Dati.beta)*femregion.h/2)*A;
end
```

```
%%%
```

```
for k = 1:length(boundary_points)
    A_0(boundary_points(k),:) = 0;
    A_0(:,boundary_points(k)) = 0;
    A_0(boundary_points(k),boundary_points(k)) = 1;
    T_0(boundary_points(k),:) = 0;
    T_0(:,boundary_points(k)) = 0;
    T_0(boundary_points(k),boundary_points(k)) = 1;
    P_0(boundary_points(k),:) = 0;
    P_0(:,boundary_points(k)) = 0;
    P_0(boundary_points(k),boundary_points(k)) = 1;
    b_0(boundary_points(k)) = 0;

end

b = b_0;
A = A_0;
P = P_0;
T = T_0;
```

Esercizio 2

Per i dettagli si vedano i file `Lab7_Es2.m` e `C_dati.m`.

- Fissata una mesh di tipo strutturato, il plot della soluzione numerica presenta oscillazioni numeriche evidenti.
- Un raffinamento della mesh comporta un miglioramento della rappresentazione dal punto di vista della stabilità, limitando le oscillazioni numeriche. Introducendo diffusività artificiale, invece, basata sul metodo *upwind*, si ricava $\alpha_h = \alpha(1 + Pe)$, da cui $Pe < 1$ e, di conseguenza, come dimostrato nella teoria degli elementi finiti in una dimensione, stabilità della soluzione agli elementi finiti. Tuttavia, pur non avendo oscillazioni nella soluzione, essa risulta sovradiffusiva.
- Si riportano di seguito gli errori calcolati ed i corrispondenti ordini di convergenza p per triangolazioni strutturate sul problema stabilizzato.

h	0.1768	0.0884	0.0442	0.0221
$\|u - u_h\|_{L^2(\Omega)}$	0.1754	0.1268	0.0796	0.0448

Tabella 1: Errore $L^2(\Omega)$ calcolato come funzione della dimensione di griglia h.

$p_{L^2(\Omega)}$	0.4680	0.6723	0.8294

Tabella 2: Ordini di convergenza $L^2(\Omega)$.

La tabella degli ordini, per mesh raffinate, conferma i risultati attesi dalla teoria, dal momento che l'applicazione del metodo upwind porta alla perdita di un ordine di accuratezza rispetto alla norma $L_2(\Omega)$.